

Practical 01

Your MLOps Workbench

A clean environment, a pinned library list,
and a run that repeats itself

Student	Lakhan Gupta
Enrolment number	S24CSEU0067
Programme	B.Tech CSE · 5th Semester
Institution	Bennett University · Session 2026-27
Course outcome	CO1 · follows lectures L01–L02
Notebook	P01_S24CSEU0067.ipynb
Self-check result	7 of 7 checks passed

SOURCE CODE REPOSITORY

https://github.com/LakhanxGupta/ML0PS_BU

https://github.com/LakhanxGupta/ML0PS_BU/tree/main/P01_lab

01 Step 0 - Check the workbench

Preflight: confirms the interpreter and that numpy, pandas and scikit-learn are importable.

```
In [6]: # Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.13.14
Folder  : P01-workbench
numpy    ok
pandas   ok
sklearn  ok
```

All good. You can carry on to Step 1.

02 Step 0b - The dataset

Confirms the shared 600-row delivery dataset is present; rebuilds it from seed 42 only if missing.

```
In [7]: # The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("../") / "data" / "delivery_times.csv"

def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path

if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found ->", DATA)

dataset found -> ../data/delivery_times.csv
```

03 Step 1 - Which Python is running this notebook?

Reports the exact interpreter running the notebook. **Inside .venv : True** confirms the course environment.

```
In [8]: import sys
from pathlib import Path

print("Python version :", sys.version.split()[0])
print("Python program :", sys.executable)

# A virtual environment is just a folder. If the path above sits
# inside a folder called .venv, you are in the course environment.
in_venv = ".venv" in sys.executable.replace("\\", "/")
print("Inside .venv :", in_venv)
print("Working folder :", Path.cwd())

Python version : 3.13.14
Python program : /Users/lakhan/Desktop/Code/SCSE3040-Lab-main/P01-workbench/.venv/bin/python
Inside .venv : True
Working folder : /Users/lakhan/Desktop/Code/SCSE3040-Lab-main/P01-workbench
```

04 Step 2 - What is installed in this environment?

Reads the installed version of each library directly from the environment.

```
In [9]: from importlib.metadata import version

LIBRARIES = ["numpy", "pandas", "scikit-learn", "matplotlib"]

for name in LIBRARIES:
    print(f"{name:<15} {version(name)}")

numpy          2.5.1
pandas         2.3.3
scikit-learn   1.9.0
matplotlib     3.11.1
```

05 Step 3 - Freeze those versions into requirements.txt

Writes those versions to work/requirements.txt, pinned with == so the environment can be rebuilt exactly.

```
In [10]: WORK = Path("work")
WORK.mkdir(exist_ok=True)

lines = [f"{name}=={version(name)}" for name in LIBRARIES]
(WORK / "requirements.txt").write_text("\n".join(lines) + "\n",
                                       encoding="utf-8")

print("wrote", WORK / "requirements.txt")
print("_" * 40)
print((WORK / "requirements.txt").read_text(encoding="utf-8"))

wrote work/requirements.txt
-----
numpy==2.5.1
pandas==2.3.3
scikit-learn==1.9.0
matplotlib==3.11.1
```

06 Step 4 - Random numbers change every time you ask

An unseeded generator: the three numbers differ on every run. This is the problem the next step fixes.

```
In [23]: import numpy as np

careless = np.random.default_rng() # no seed given
print("three random numbers:", np.round(careless.uniform(0, 10, 3), 2))

three random numbers: [2.56 1.63 9.55]
```

07 Step 5 - A seed makes the randomness repeat

Two generators given the same seed produce identical sequences. **identical : True.**

```
In [12]: first = np.random.default_rng(42).uniform(0, 10, 3)
second = np.random.default_rng(42).uniform(0, 10, 3)

print("first run :", np.round(first, 2))
print("second run:", np.round(second, 2))
print("identical :", np.array_equal(first, second))

first run : [7.74 4.39 8.59]
second run: [7.74 4.39 8.59]
identical : True
```

08 Step 6 - Build the delivery dataset, twice

The dataset is built twice into separate files; matching SHA-256 checksums prove the two runs are byte-identical.

```
In [13]: import hashlib

def sha256_of(path):
    """A short fingerprint of a file's exact contents."""
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

make_delivery_csv(WORK / "run_a.csv")
make_delivery_csv(WORK / "run_b.csv")

fp_a = sha256_of(WORK / "run_a.csv")
fp_b = sha256_of(WORK / "run_b.csv")

print("run A:", fp_a[:16], "...")
print("run B:", fp_b[:16], "...")
print("identical files:", fp_a == fp_b)

run A: 9e9f7a46c817d5bb ...
run B: 9e9f7a46c817d5bb ...
identical files: True
```

09 Step 7 - Look at the data you just made

Loads the CSV and reports its shape, first rows and summary statistics.

```
In [14]: import pandas as pd

orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
print()
print(orders.describe().round(1))

rows, columns: (600, 5)

   distance_km  prep_time_min  traffic_level  rain  delivery_min
0         9.40             17              1     0          51.3
1         5.55             24              2     1          54.2
2        10.37             28              3     0          67.7
3         8.52             23              2     0          51.2
4         1.58             29              2     1          42.1

   distance_km  prep_time_min  traffic_level  rain  delivery_min
count      600.0         600.0         600.0  600.0         600.0
mean         6.2          17.6           2.0    0.3          46.6
std          3.3           7.4           0.8    0.4          12.1
min           0.6           5.0           1.0    0.0          18.5
25%           3.2          11.0           1.0    0.0          37.8
50%           6.2          17.0           2.0    0.0          46.5
75%           9.1          24.0           3.0    1.0          55.7
max          12.0          30.0           3.0    1.0          75.1
```

10 Step 8 - Record what you ran

Writes work/run_info.json recording the Python version, seed, row count, data checksum and library versions.

```
In [15]: import json

run_info = {
    "python": sys.version.split()[0],
    "seed": SEED,
    "rows": len(orders),
    "data_sha256": sha256_of(DATA),
    "libraries": {n: version(n) for n in LIBRARIES},
}

(WORK / "run_info.json").write_text(json.dumps(run_info, indent=2),
                                     encoding="utf-8")
print(json.dumps(run_info, indent=2))

{
  "python": "3.13.14",
  "seed": 42,
  "rows": 600,
  "data_sha256": "61bfd2533f407c8ae8a1adcb6e42e1f0fe42cf9c8a6bfe8747673abcafc828c4",
  "libraries": {
    "numpy": "2.5.1",
    "pandas": "2.3.3",
    "scikit-learn": "1.9.0",
    "matplotlib": "3.11.1"
  }
}
```

11 Step 9 - Save your work in git

Initialises a git repository inside work/ and commits the pinned requirements and the run record.

```
In [24]: import subprocess

def git(*args):
    """Run one git command inside work/ and show what it said."""
    done = subprocess.run(["git", *args], cwd=WORK,
                          capture_output=True, text=True)
    print("$ git", " ".join(args))
    print((done.stdout + done.stderr).strip() or "(no output)")
    print("-" * 50)
    return done

if not (WORK / ".git").exists():
    git("init", "-q")
    git("config", "user.name", "LakhanxGupta")
    git("config", "user.email", "mail2guptalakhan@gmail.com")
    git("add", "requirements.txt", "run_info.json")
    git("commit", "-q", "-m", "P01: pinned requirements and run record")
    git("log", "--oneline")

$ git init -q
(no output)

-----
$ git config user.name LakhanxGupta
(no output)

-----
$ git config user.email mail2guptalakhan@gmail.com
(no output)

-----
$ git add requirements.txt run_info.json
(no output)

-----
$ git commit -q -m P01: pinned requirements and run record
(no output)

-----
$ git log --oneline
6551000 P01: pinned requirements and run record

-----
Out[24]: CompletedProcess(args=['git', 'log', '--oneline'], returncode=0, stdout='6551000 P01: pinned requirements and run record\n', stderr='')
```

12 Task T1 - Change the seed

Task T1. The distance column rebuilt with seed 7 in place of 42, giving [7.69, 10.82, 9.42].

```
In [25]: # T1 --- build the distance column with seed 7 instead of 42.
# Same draw as make_delivery_csv: 600 uniforms between 0.5 and 12.0,
# rounded to 2 decimals. A fresh generator, so we take from the front.
T1_first_three = list(np.round(np.random.default_rng(7).uniform(0.5, 12.0, 600), 2)[:3])

print("T1_first_three =", T1_first_three)

T1_first_three = [np.float64(7.69), np.float64(10.82), np.float64(9.42)]
```

13 Task T2 - Write your own pinned requirements file

Task T2. Writes work/my_requirements.txt, three libraries pinned to the versions read from this machine.

```
In [26]: MY_LIBS = ["numpy", "pandas", "scikit-learn"]

# Ask Python for each version, so the file is right on any machine.
# Note: "scikit-learn" is the install name; version("sklearn") would fail.
my_lines = [f"{name}=={version(name)}" for name in MY_LIBS]
(WORK / "my_requirements.txt").write_text("\n".join(my_lines) + "\n",
                                          encoding="utf-8")

print(my_lines)

['numpy==2.5.1', 'pandas==2.3.3', 'scikit-learn==1.9.0']
```

14 Task T3 - Fingerprint a run

Task T3. fingerprint() returns the row count, SHA-256 checksum and seed for a given CSV.

```
In [27]: def fingerprint(path):
        """Three facts that identify one run: size, contents, and seed."""
        return {
            "rows": len(pd.read_csv(path)), # header is not counted
            "sha256": sha256_of(path),      # helper from Step 6
            "seed": SEED,                   # the course seed, 42
        }

T3_fp = fingerprint(DATA)
print(T3_fp)

{'rows': 600, 'sha256': '61bfd2533f407c8ae8a1adcb6e42e1f0fe42cf9c8a6bfe8747673abcafc828c4', 'seed': 42}
```

15 Self-check - marking table

Self-check: 7 of 7 checks passed.

```
In [28]: # -----
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# -----

_results = []

def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))

_check('T1 | T1_first_three holds three numbers', lambda: len(T1_first_three) == 3)
_check('T1 | those are the first three distances for seed 7', lambda: all(abs(float(a) - float(b)) < 1e-9 for a, b in zip(T1_first_three, T1_distances)))
_check('T2 | work/my_requirements.txt exists', lambda: (WORK / 'my_requirements.txt').is_file())
_check('T2 | it pins all three libraries with ==', lambda: sum(1 for ln in (WORK / 'my_requirements.txt').read_text(encoding='utf-8').splitlines() if '==' in ln) == 3)
_check('T3 | fingerprint() returns the three required keys', lambda: set(T3_fp) == {'rows', 'sha256', 'seed'})
_check('T3 | it counts 600 data rows and uses seed 42', lambda: T3_fp['rows'] == 600 and T3_fp['seed'] == 42)
_check('T3 | the checksum matches the file on disk', lambda: T3_fp['sha256'] == sha256_of(DATA) and len(T3_fp['sha256']) == 64)

print("=====")
print("SELF-CHECK   Practical 01 --- Your MLOps Workbench")
print("=====")
for _label, _ok in _results:
    print(f" [{ 'PASS' if _ok else 'FAIL' }]  {_label}")
print("=====")
_passed = sum(1 for _, _ok in _results if _ok)
print(f" { _passed} of {len(_results)} checks passed")
print("=====")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")

=====
SELF-CHECK   Practical 01 --- Your MLOps Workbench
=====
[PASS] T1 | T1_first_three holds three numbers
[PASS] T1 | those are the first three distances for seed 7
[PASS] T2 | work/my_requirements.txt exists
[PASS] T2 | it pins all three libraries with ==
[PASS] T3 | fingerprint() returns the three required keys
[PASS] T3 | it counts 600 data rows and uses seed 42
[PASS] T3 | the checksum matches the file on disk
=====
7 of 7 checks passed
=====
Well done. Save the notebook and submit it.
```